

Day 1 Cheatsheet

Basic R

Major concepts

- **Package** - a package in R is a bundle or “package” of code (and or possibly data) that can be loaded together for easy repeated use or for sharing with others. Like an “expansion pack”.
- **Function** - a function is a particular piece of code that allows you to do something in R. You can write your own, use functions that come directly from installing R, or use functions from additional packages.
- **Argument** - an option that you specify to a function.
- **Object** - an object is something that can be worked with in R - can be lots of different things!
- **Tidyverse** - This is a newer set of packages designed for data science that can make your code more intuitive as compared to the original older Base R.
- The R console is a full calculator:
 - +, -, /, * are add, subtract, divide and multiply
 - ^ or ** is power
 - parentheses – (and) – work with order of operations
 - %% finds the remainder
- # is the comment symbol; nothing to its right is evaluated.

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<-	x <- 1	Assigns a name x to something in the R environment.
Base R	c()	x <- c(1, 3)	Combines values into a vector.
Base R	str()	str(x)	Gets a summary of the object x structure.
Base R	class()	class(x)	Returns the type of the values in object x.
Base R	length()	length(x)	Returns how long the object x is.
Base R	seq()	seq(from = 0, to = 100, by = 5)	Generate regular sequences.
Base R	rep()	rep(1, times = 10)	Replicates the values in x. Can take times or length.out argument.

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>sample()</code>	<code>sample(1:12)</code>	Takes a sample of the specified size from the elements of x using either with or without replacement. <code>replace = TRUE</code> samples with replacement.
Base R	<code>install.packages()</code>	<code>install.packages("tidyverse")</code>	Installs packages
Base R	<code>library()</code>	<code>library(tidyverse)</code>	Loads and attaches additional packages to the R environment. Done every time you start R.

RStudio

Major concepts

- **RStudio** - an Integrated Development Environment (IDE) for R - makes it easier to use R.
- **Source/Editor** - “Analysis” Script + Interactive Exploration - In a .R file (we call a script), code is saved on your disk
- **R Console** - Where code is executed (where things happen) - Code is not saved on your disk
- **Workspace/Environment** - Tells you what objects are in R. What exists in memory/what is loaded?/what did I read in?
- **R Markdown** - Files (.Rmd) help generate reports that include your code and output.
- **R Project** - Helps you organize your work. Helps with working directories (discussed later). Allows you to easily know which project you’re on.
- RStudio Keyboard shortcuts

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>View()</code>	<code>View(mtcars)</code>	allows you to view data in a spreadsheet format.
Base R	<code>head()</code>	<code>head(mtcars)</code>	Returns the top 6 rows of an object in the environment by default. You can specify how many rows you want by including the <code>n</code> =argument.

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>tail()</code>	<code>tail(mtcars)</code>	Returns the bottom 6 rows of an object in the environment by default. You can specify how many rows you want by including the <code>n =</code> argument.

* This format was adapted from the cheatsheet format from AlexsLemonade.

Day 2 Cheatsheet

Reproducibility

Major concepts

- **Reproducibility** - A different analyst re-performs the analysis with the same code and the same data and obtains the same result.
- **Repeatable** - keeping everything the same but repeating the analysis - do we get the same results
- **Reproducible** - using the same data and analysis but in the hands of another researcher - do we get the same results?
- **Replicable** - with new data do we obtain the same inferences?

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>sessionInfo()</code>	<code>sessionInfo()</code>	Returns the R version information, the OS, and the attached packages in the current R session.

More resources

- The RMarkdown book
- Jenny Bryan's organizational strategies.
- Write efficient R code for science.
- Reproducibility in Cancer Informatics course

Data Input/Output

Major concepts

- **Delimited file** - columns within the file have boundaries created with some type of punctuation - for example, a csv file stands for comma separated values, thus the columns are delimited or separated by commas. txt files often use tabs.
- **tibble** - a rectangular data frame, where data are split into rows and columns.
- **File path** - where a file lives on your computer (or on the internet) - File paths can be relative or absolute.
- **Getting help** - For any function, you can write `?FUNCTION_NAME`, or `help("FUNCTION_NAME")` to look at the help file for that function.
- **R Projects** set the working directory where the .Rproj file is.
- **Reading in data (manual)** - *Download the data, Put data in the project folder.* Go to File, Import Dataset, From Text (**readr**), browse for the file, and finally click "Update" and "Import".

Functions

Library/Package	Piece of code	Example of usage	What it does
<code>tidyverse (readr)</code>	<code>read_csv()</code>	<code>df <- read_csv("<url>")</code>	Reads in a CSV file from a specified file path.
<code>tidyverse (readr)</code>	<code>read_tsv()</code>	<code>df <- read_tsv("<url>")</code>	Reads in a TSV file from a specified file path.
<code>tidyverse (readr)</code>	<code>read_delim()</code>	<code>df <- read_delim("<url>", delim = ":")</code>	Reads in a delimited file from a specified file path.
Base R	<code>head()</code> / <code>tail()</code>	<code>head(df)</code> / <code>tail(df)</code>	Returns the first or last part of a dataset (or other object).
Base R	<code>getwd()</code>	<code>getwd()</code>	Finds the current working directory.
Base R	<code>setwd()</code>	<code>setwd("Desktop")</code>	Changes the current working directory.
Base R	<code>View()</code>	<code>View(df)</code>	View a spreadsheet-style data viewer on a matrix-like R object.
Base R	<code>str()</code>	<code>str(df)</code>	Display the internal structure of an R object (dimensions, class, etc).

* This format was adapted from the cheatsheet format from AlexsLemonade.

Day 3 Cheatsheet

Subsetting Data in R

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>nrow(x); ncol(x)</code>	<code>nrow(x); ncol(x)</code>	Get the number of rows and the number of columns in an object <code>x</code> , respectively.
Base R	<code>dim(x)</code>	<code>dim(x)</code>	Get the number of rows <i>and</i> number of columns in an object <code>x</code>
dplyr	<code>glimpse(x)</code>	<code>glimpse(mtcars)</code>	Get an overview of data frame <code>x</code>
dplyr	<code>slice_sample(x)</code>	<code>slice_sample(mtcars)</code>	See a random subset of the rows of <code>x</code>
Base R	<code>data.frame()</code>	<code>df <- data.frame(1:3)</code>	Creates a data frame where the named arguments will be the same length.
Base R	<code>tibble()</code>	<code>tibble(mtcars)</code>	Creates a tibble from a data.frame or matrix.
tibble	<code>column_to_rownames()</code>	<code>df <- df %>% column_to_rownames('old_col')</code>	Transforms an existing column called by a string into the rownames.
tibble	<code>rownames_to_column()</code>	<code>df <- df %>% column_to_rownames('new_col')</code>	Transforms the rownames of a data frame into a column (which is added to the start of the data frame). The string supplied as an argument will be the name of the new column.
dplyr	<code>rename()</code>	<code>df <- rename(df, MPG = mpg)</code>	Renames designated columns while keeping all variables of the data.frame
dplyr	<code>pull()</code>	<code>pull(df, 'existing_variable_name')</code>	Extract a column as a vector
dplyr	<code>select()</code>	<code>select(df, 'existing_variable_name')</code>	Selects columns that match the specified argument

Library/Package	Piece of code	Example of usage	What it does
dplyr	<code>filter()</code>	<code>filter(df, mpg > 20)</code>	Returns a subset of rows matching the conditions of the specified logical argument
Base R	<code>==, <=, >=, !=</code>	<code>filter(df, mpg > 20)</code>	These are binary operators which allow for the comparison of values in an object. They are handy for use with <code>filter()</code>
Base R	<code>%in%</code>	<code>filter(df, mpg %in% c(20,21,22))</code>	Checks if the given value(s) on the left side of the operator are in the vector or other R object defined on the right side of the operator. It returns a logical <code>TRUE</code> or <code>FALSE</code> statement.
dplyr	<code>%>%</code>	<code>df <- df %>% select('new_variable_name')</code>	Funnel a data.frame through tidyverse operations
dplyr	<code>mutate()</code>	<code>df <- mutate(df, newcol = wt/2.2)</code>	Adds a new column that is a function of existing columns
dplyr	<code>relocate()</code>	<code>df_carb <- relocate(.data = df, wt, .before = mpg)</code>	Reorder columns in a data frame or tibble
dplyr	<code>arrange()</code>	<code>df <- arrange(df, mpg)</code>	Reorders rows in ascending order. <code>arrange(desc())</code> would reorder rows in descending order.
dplyr	<code>case_when()</code>	<code>df <- arrange(df, mpg)</code>	This function allows you to vectorize multiple <code>if_else()</code> statements. If no cases match, <code>NA</code> is returned.
Base R	<code>colnames()</code>	<code>colnames(df)</code>	Gets or sets the column names of a matrix or data frame.

- See `tidyselect` helpers for handy things to use with `select()`.

* This format was adapted from the cheatsheet format from AlexsLemonade.

Day 4 Cheatsheet

Data Summarization

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>min(x)</code>	<code>min(x)</code>	Returns the minimum value of all values in an object <code>x</code> .
Base R	<code>sum(x)</code>	<code>sum(x)</code>	Returns the sum of all values (values must be integer, numeric, or logical) in object <code>x</code> .
Base R	<code>mean(x)</code>	<code>mean(x)</code>	Returns the arithmetic mean of all values (values must be integer or numeric) in object <code>x</code> or logical vector <code>x</code> .
Base R	<code>log(x)</code>	<code>log(x)</code>	Gives the natural logarithm of object <code>x</code> . <code>log2(x)</code> can be used to give the logarithm of the object in base 2. Or the base can be specified as an argument.
Base R	<code>range(x)</code>	<code>range(x)</code>	Gives the min and max for object <code>x</code> .
Base R	<code>sd(x)</code>	<code>sd(x)</code>	Gives the standard deviation for object <code>x</code> .
Base R	<code>sqrt(x)</code>	<code>sqrt(x)</code>	Gives the square root for object <code>x</code> .
Base R	<code>quantile(x)</code>	<code>quantile(x, probs = .5)</code>	Produces sample quantiles corresponding to the given probabilities <code>x</code> .
Base R	<code>summary(x)</code>	<code>summary(x)</code>	Returns a summary of the values in object <code>x</code> .
dplyr	<code>pull()</code>	<code>x_vect <- df %>% pull(x)</code>	Extract a single column into vector form. <code>pull()</code> is very handy before summary functions like <code>mean()</code> , <code>sum()</code> , etc.

Library/Package	Piece of code	Example of usage	What it does
dplyr	summarize()	df <- df %>% summarize(mean_x = mean(x))	Summarizes multiple values in an object into a single value. This function can be used with other functions to retrieve a single output value for the grouped values. summarize and summarise are synonyms in this package. However, note that this function does not work in the same manner as the base R summary function.
dplyr	distinct()	df %>% distinct(factor_name)	Display unique/distinct rows from a data frame or tibble
dplyr	n_distinct()	x_vect %>% n_distinct()	Counts the number of unique/distinct combinations in a set of one or more vectors.
dplyr	count()	df %>% count(factor_name)	Count the number of groups in a factor variable of a data frame or tibble
dplyr	group_by()	df %>% group_by(factor_name)	Groups data into rows that contain the same specified value(s)
dplyr	ungroup()	df %>% ungroup()	Undo a grouping that was done by group_by()
Base R	unique()	unique(df)	Returns a vector, data frame or array like x but with duplicate elements/rows removed.
Base R	rowSums()	rowSums(df)	Calculates sums for each row
Base R	colSums()	colSums(df)	Calculates sums for each column
Base R	rowMeans()	rowMeans(df)	Calculates means for each row
Base R	colMeans()	colMeans(df)	Calculates means for each column

- Many summarizing functions (e.g., **mean()**, **sum()**) have the argument **na.rm = TRUE**. This can be used to ignore missing data.

Data Classes

Major concepts

- **Character** - strings or individual characters, quoted
- **Numeric** - any real number(s)
- **Double** - a special subset of numeric that contains fractional values.
- **Integer** - any integer(s)/whole numbers
- **Factor** - categorical/qualitative variables
- **Logical** - variables composed of TRUE or FALSE
- **Date/POSIXct** - represents calendar dates and times
- **matrix** - Two-dimensional class of data where all rows and columns consist of the same data type.
- **data frame** - Two-dimensional class of data where all columns can be of different data types.
- **list** - Can be of varying dimensions and can hold any kind of data type. Can hold vectors, strings, matrices, models, list of other lists.

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>as.numeric(x)</code>	<code>as.numeric(x)</code>	Coerces object x into numeric class. This type of function can be used to coerce object x into other data types, i.e., <code>as.character</code> , <code>as.numeric</code> , <code>as.data.frame</code> , <code>as.matrix</code> , <code>as.Date</code> etc.
lubridate	<code>ymd(x)</code>	<code>ymd("2024-01-31")</code>	Coerces character object x into date class. The format of the character object determines the function to use. Other examples include <code>mdy()</code> , <code>dmy()</code> , etc.

- `lubridate` is a powerful, widely used R package from “tidyverse” family to work with Date / POSIXct class objects

* This format was adapted from the cheatsheet format from AlexsLemonade.

Day 5 Cheatsheet

Data Cleaning

Major concepts

- **Most important rule of data handling - Always be looking at your data!**
- **NA** - general missing data
- **NaN** - stands for “Not a Number”, happens when you do 0/0.
- **Inf** and **-Inf** - Infinity, happens when you take a positive number (or negative number) by 0.

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>is.na(x)</code>	<code>is.na(x)</code>	checks if x is NA .
Base R	<code>is.nan(x)</code>	<code>is.nan(x)</code>	checks if x is NaN .
Base R	<code>is.infinite(x)</code>	<code>is.infinite(x)</code>	checks if x is Inf or -Inf .
naniar	<code>pct_complete(x)</code>	<code>pct_complete(x)</code>	Reports the percentage of data that is complete in x .
naniar	<code>gg_miss_var(x)</code>	<code>gg_miss_var(x)</code>	Reports as a plot the percentage of data that is complete in x .
tidyr	<code>drop_na(df)</code>	<code>drop_na(df)</code>	Drops rows of NA from a given data frame/tibble
dplyr	<code>case_when()</code>	<code>df <- arrange(df, mpg)</code>	This function allows you to vectorize multiple <code>if_else()</code> statements. If no cases match, NA is returned.
dplyr	<code>mutate()</code>	<code>df <- mutate(df, newcol = wt/2.2)</code>	Adds a new column that is a function of existing columns
dplyr	<code>separate()</code>	<code>df %>% separate(x, c("A", "B"))</code>	Separate a character column into multiple columns with a regular expression or numeric locations
dplyr	<code>unite()</code>	<code>df %>% unite("z", x:y, remove = FALSE)</code>	Unite multiple columns together into one column

Library/Package	Piece of code	Example of usage	What it does
<code>stringr</code>	<code>str_detect</code>	<code>df %>% filter(str_detect(col_name, "string_pattern"))</code>	Returns logical vector to indicate if string pattern was detected
<code>stringr</code>	<code>str_replace</code>	<code>str_replace(vector, "replace_me","with_me")</code>	Replaces all instances of one specified string with another specified string

* This format was adapted from the cheatsheet format from AlexsLemonade.

Day 6 Cheatsheet

Data Manipulation

Major concepts:

- Wide data - multiple columns per individual, values spread across multiple columns
- Long data - multiple rows per observation, a single column contains the values

Functions

Library/Package	Piece of code	Example of usage	What it does
dplyr	separate()	df %>% separate(x, c("A", "B"))	Separate a character column into multiple columns with a regular expression or numeric locations
dplyr	unite()	df %>% unite("z", x:y, remove = FALSE)	Unite multiple columns together into one column
tidyr	pivot_longer()	df %>% pivot_longer(!col_to_keep, names_to = "new_col_with_labels", values_to = "new_col_with_values")	Lengthens a data frame by increasing the number of rows and decreasing the number of columns.
tidyr	pivot_wider()	df %>% pivot_wider(names_from = "col_with_names", values_from = "col_with_values")	Widens a data frame by decreasing the number of rows and increasing the number of columns.
dplyr	?_join()	inner_join(x, y)	Joins data from two data frames. inner_join - only rows that match for x and y are kept. full_join - all rows of x and y are kept. left_join - all rows of x are kept even if not merged with y. right_join - all rows of y are kept even if not merged with x. anti_join - all rows from x not in y keeping just columns from x.

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>uplicated()</code>	<code>uplicated(x)</code>	Determines and removes duplicate elements from <code>x</code> .
Base R	<code>t()</code>	Transpose	Returns the transpose of a matrix or data frame. If given a data frame, returns a matrix.

Data Visualization with **esquisse**

Major concepts

- The **esquisse** package can help you to test out plots and get familiar with the syntax required to make plots

Functions

Library/Package	Piece of code	Example of usage	What it does
esquisse	<code>esquisser()</code>	<code>esquisser(Orange)</code>	Start an esquisse interactive session to create a plot of the given <code>data.frame</code> or <code>tibble</code>

* This format was adapted from the cheatsheet format from AlexsLemonade.

Day 7 Cheatsheet

Data Visualization

Major concepts:

- The **ggplot2** package is a tidyverse package for producing graphics.
- Tidy data: Each variable forms a column and each observation forms a row.
- Having data in tidy format and long format makes creating plots easier!
- Remember to use a **+** at the end of each line when adding new layers to plots.

Pipes like **%>%** do not work to add layers to plots, but can be used to pipe data into plots.

This works: `ggplot(data = iris, aes(x = Species, y = Petal.Length)) + geom_boxplot()`

This works: `iris %>% ggplot(aes(x = Species, y = Petal.Length)) + geom_boxplot()`

This doesn't: `iris %>% ggplot(aes(x = Species, y = Petal.Length)) %>% geom_plot()`

- Factor: A factor is a special character vector where the elements have pre-defined groups or “levels”. You can think of these as qualitative or categorical variables. An example is 1st grade, 2nd grade, etc. It is important to specify factors as the class **factor** so that R recognizes it as such.

Functions

Library/Package	Piece of code	Example of usage	What it does
ggplot2	<code>ggplot()</code>	<code>ggplot(data = iris)</code>	Begins a plot that is finished by adding layers.
ggplot2	<code>aes()</code>	<code>ggplot(data = iris, aes(x = Species, y = Petal.Length))</code>	Aesthetic Mappings - Designates how variables in the data object will be mapped to the visual properties of the ggplot.
ggplot2	<code>geom_boxplot()</code>	<code>ggplot(data = iris, aes(x = Species, y = Petal.Length)) + geom_boxplot()</code>	Creates a boxplot when added as a layer to a <code>ggplot()</code> object.
ggplot2	<code>geom_density()</code>	<code>ggplot(data = iris, aes(x = Petal.Length)) + geom_density()</code>	Creates a smoothed density plot when added as a layer to a <code>ggplot()</code> object based on the computed density estimate.
ggplot2	<code>geom_point()</code>	<code>ggplot(data = iris, aes(x = Species, y = Petal.Length)) + geom_point()</code>	Creates a scatterplot when added as a layer to a <code>ggplot()</code> object.

Library/Package	Piece of code	Example of usage	What it does
ggplot2	geom_line()	ggplot(data = iris, aes(x = Species, y = Petal.Length)) + geom_line()	Creates a line plot when added as a layer to a ggplot() object by connecting the points in order of the x axis variable.
ggplot2	geom_hline()	ggplot(data = iris, aes(x = Species, y = Petal.Length)) + geom_point() + geom_hline(yintercept = 4)	Annotates a plot with horizontal lines when added as a layer to a ggplot() object with one of the geom functions used to draw the plot, for example, geom_point() .
ggplot2	theme_classic()	ggplot(data = iris, aes(x = Petal.Length)) + geom_density() + theme_classic()	Displays ggplot without grid lines. The ggtheme documentation has descriptions on additional themes that can be used.
ggplot2	xlab(); ylab(); ggtitle()	ggplot(data = iris, aes(x = Petal.Length)) + geom_density() + xlab("Petal Length")	Modifies the labels on the x axis and on the y axis, and sets the title of a ggplot , respectively.
ggplot2	facet_wrap()	ggplot(data = iris, aes(x = Petal.Length)) + geom_density() + facet_wrap(~Species)	Plots individual graphs using specified variables to subset the data.
ggplot2	ggsave()	ggsave(filename = "plotname.pdf")	Saves the last plot in working directory.
ggplot2	last_plot()	Last plot()	Returns the last plot produced.

[Click here for a summary of the ggplot2 theme system~](#)

Factors

Library/Package	Piece of code	Example of usage	What it does
base	<code>factor()</code>	<code>fact_qual <-factor(c("poor", "fine", "good"))</code>	Creates a factor out of a vector.
base	<code>levels()</code>	<code>levels(fact_qual)</code>	Shows the levels (and order) of a factor vector.
base	<code>levels()</code>	<code>levels(fact_qual) <- c("poor", "fine", "good")</code>	Allows the order of the levels of a factor to be changed.
forcats	<code>fct_reorder()</code>	<code>fct_reorder(.f = grade, .x = absences)</code>	Allows the order of the levels of a factor variable (specified by the <code>.f</code> argument) to be changed according to another variable (specified by the <code>.x</code> argument). In this case grade is ordered by absences. This is especially helpful for plots or statistical output!

* This format was adapted from the cheatsheet format from AlexsLemonade.

Day 8 Cheatsheet

Statistics

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>cor(x, y)</code>	<code>cor(x, y)</code>	Calculate correlation of two vectors in R.
<code>corrplot</code>	<code>corrplot(cor_mat)</code>	<code>corrplot(cor_mat, type = "upper", order = "hclust")</code>	Create a correlation matrix plot.
Base R	<code>t.test(x, y)</code>	<code>t.test(x, y, alternative = "two.sided")</code>	Perform one and two sided t tests.
<code>broom</code>	<code>tidy(x)</code>	<code>tidy(t_test_result)</code>	Manipulates and tidies up wonky statistical R objects into nice data frames
Base R	<code>wilcox.test(x, y)</code>	<code>wilcox.test(x, y)</code>	Calculate non-parametric, Wilcoxon signed rank test, Wilcoxon rank sum test.
Base R	<code>shapiro.test()</code>	<code>shapiro.test(x)</code>	Test for normality with Shapiro-Wilk.
Base R	<code>ks.test()</code>	<code>ks.test(x)</code>	Test for normality with Kolmogorov-Smirnov.
Base R	<code>var.test(x, y)</code>	<code>var.test(x, y)</code>	Compare two variances with Fisher's F-test
Base R	<code>chisq.test(x, y)</code>	<code>chisq.test(x, y)</code>	Perform chi squared contingency tables and goodness of fit tests
Base R	<code>lm(x ~ y)</code>	<code>lm(x ~ y, data = df)</code>	Fit linear models based on a formula you provide.
Base R	<code>summary(x)</code>	<code>summary(linear_model_result)</code>	Returns a summary of the values in object, including a linear model or other statistical test.

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>glm(x ~ y)</code>	<code>glm(x ~ y, data = df, family = binomial())</code>	Fit generalized linear models based on a formula you provide. Must specify the error distribution and link function using the <code>family</code> argument.

* This format was adapted from the cheatsheet format from AlexsLemonade.

Day 9 Cheatsheet

Functions

Why create your own functions? - Cut down on repetitive code (easier to fix things!) - Organize code into manageable chunks - Avoid running code unintentionally - Use names that make sense to you

Syntax for a function is:

```
functionName <- function(inputs) {  
  # Document your inputs here  
  <function body>  
  return(value)  
}
```

Functions

Library/Package	Piece of code	Example of usage	What it does
Base R	<code>lapply()</code>	<code>lapply(some_list, a_function)</code>	Apply a function over a list or vector.
Base R	<code>sapply()</code>	<code>sapply(some_list, a_function)</code>	Apply a function over a list or vector but by default returns a matrix or vector.

* This format was adapted from the cheatsheet format from AlexsLemonade.